

1. Github REST API

Loại kiến trúc:

Base URL: <https://api.github.com>

Auth method:

- Unauthenticated (rate limit thấp)
- Personal access token (PAT) / OAuth 2.0 Token gửi qua header
Authorization: Bearer <Token>

Versioning: Header-based versioning qua custom header:

X-GitHub-API-Version: 2026-03-10

Resource Identifier: Linh hoạt giữa số nguyên (integer) và chuỗi (string)

- + Issue/Pull request: Số nguyên tuần tự (1,42)
- + Repository: Chuỗi {owner}/{repo} (VD: Aquaneee/HK1_2627_INT3505E_2) hoặc integer ID
- + User: username hoặc ID

2. Spotify Web API

Loại kiến trúc: REST

Base URL: <https://api.spotify.com/v1>

Auth method:

- OAuth 2.0 (Client Credentials Flow, Authorization Code Flow) gửi qua header:
Authorization: Bearer <access_token>

Versioning: URI Path versioning (chỉ định phiên bản /v1 cố định trực tiếp trong Base URL)

Resource Identifier: Chuỗi ký tự Base62 gồm 22 ký tự (VD: 4iV5W9uYEdYUVa79Axb7Rh), không phải số nguyên tuần tự hay chuẩn UUID 36 ký tự

3. OpenWeatherMap API

Loại kiến trúc: REST

Base URL: <https://api.openweathermap.org/data/3.0>

Auth method:

- API Key truyền qua Query Parameter: ?appid={API_KEY} hoặc gửi qua header

Versioning: URI Path versioning (/data/2.5 hoặc /data/3.0 trực tiếp trên URL path)

Resource Identifier: Số nguyên (City ID, VD: 1581130 cho Hà Nội), hoặc kết hợp các thuộc tính truy vấn như tọa độ địa lý (lat/lon) và tên thành phố (q={city_name})

So sánh 5 đặc điểm của một API tốt trong Higginbotham Chương 1 và Geewax Chương 1 với 5 đặc điểm có trong slide 5:

- Định nghĩa: API giống như một cái menu tại nhà hàng: nó ghi rõ danh sách các món ăn (chức năng) và giá tiền (dữ liệu cần truyền vào) để gọi món, giúp lấy được thứ mình muốn mà không cần bước chân vào bếp xem đầu bếp nấu nướng ra sao.
- Resource chính là món ăn cụ thể trên menu đó (ví dụ: Người dùng, Đơn hàng, Bài viết). Mỗi món đều có một tên gọi hoặc mã định danh riêng có thể xem, thêm mới, sửa hoặc xóa.
- Đối chiếu với 5 đặc điểm cốt lõi trong slide:
 - + Có hợp đồng rõ ràng (Contract-first): Cả hai tác giả đều đồng thuận sâu sắc, coi API là hợp đồng giữa bên cung cấp và bên tiêu thụ.
 - + Ẩn cài đặt (Abstraction): Hoàn toàn đồng thuận; che giấu kiến trúc bên dưới (ngôn ngữ, DB, infra).
 - + Có thể tái sử dụng (Reusability): Đồng thuận, đồng thời nâng tầm thành tư duy API như một sản phẩm số (API as a Product).
 - + Độc lập ngôn ngữ & nền tảng (Language-agnostic): Đồng thuận dựa trên các chuẩn giao thức mở (HTTP, JSON, Protobuf).
 - + Có thể versioning & quan sát được (Versioned & Observable): Đồng thuận về vai trò sống còn của việc theo dõi và tiến hóa API.
- Điểm Geewax và Higginbotham bổ sung nổi bật:
 - + Bổ sung quy trình ADR (Align, Define, Design, Review) từ Higginbotham: Thiết kế bắt đầu từ sự thống nhất mục tiêu nghiệp vụ trước khi đi vào đặc tả kỹ thuật.
 - + Bổ sung hệ thống mẫu thiết kế chuẩn hóa (Design Patterns) từ Geewax: Giúp API có tính nhất quán cao ở từng chi tiết nhỏ.
 - + Tách rời triệt để API Resource khỏi Database Schema: API model đại diện cho Domain Capabilities chứ không phải phép chiếu 1-1 của bảng DB.
 - + Tư duy tiến hóa không phá vỡ (Non-breaking Evolution): Ưu tiên các thay đổi mở rộng an toàn hơn là việc lạm dụng tăng số version (/v1, /v2).

Code bài 6 sau khi mở rộng:

```

1  from flask import Flask, jsonify, request
2  app = Flask(__name__)
3  app.json.ensure_ascii = False
4  _next = 3
5  ✓ BOOKS = [
6      {"id": 1, "title": "Clean Code", "author": "R. Martin", "year": 2008},
7      {
8          "id": 2,
9          "title": "The Pragmatic Programmer",
10         "author": "Andy Hunt",
11         "year": 1999,
12     },
13 ]
14 def find(bid):
15     return next((b for b in BOOKS if b["id"] == bid), None)
16     # (a) Tìm kiếm ?q=... ; (b) Sắp xếp ?sort=title
17     @app.route("/books", methods=["GET"])
18     ✓ def list_books():
19         res = list(BOOKS)
20         # (a) Tìm kiếm theo query ?q=... (không phân biệt hoa/thường trong title hoặc author)
21         q = request.args.get("q", "").strip().lower()
22         if q:
23             res = [
24                 b
25                 for b in res
26                 if q in b["title"].lower() or q in b["author"].lower()
27             ]
28         # (b) Sort theo trường chỉ định (hỗ trợ ?sort=title hoặc ?sort=year)
29         sort_by = request.args.get("sort")
30         if sort_by in ("title", "year", "author"):
31             res = sorted(
32                 res,
33                 key=lambda x: (
34                     x[sort_by].lower()
35                     if isinstance(x[sort_by], str)
36                     else x[sort_by]
37                 ),
38             )
39         # Giới hạn số lượng (mặc định 100)
40         n = int(request.args.get("limit", 100))
41         return jsonify(res[:n]), 200
42     # DETAIL – GET /books/<int:bid>
43     @app.route("/books/<int:bid>", methods=["GET"])
44     ✓ def get_book(bid):
45         book = find(bid)
46         if not book:
47             return jsonify({"error": "not found"}), 404
48         return jsonify(book), 200

```

```

49     # CREATE – POST /books
50     # (c) Bắt buộc field year là số >= 1900
51     @app.route("/books", methods=["POST"])
52     def create_book():
53         global _next
54         body = request.get_json(silent=True) or {}
55         t, a = body.get("title"), body.get("author")
56         y = body.get("year")
57         if not t or not a:
58             return jsonify({"error": "need title+author"}), 400
59         # Kiểm tra ràng buộc year
60         if y is None or not isinstance(y, int) or y < 1900:
61             return jsonify({"error": "year must be an integer >= 1900"}), 400
62         book = {"id": _next, "title": t, "author": a, "year": y}
63         _next += 1
64         BOOKS.append(book)
65         return jsonify(book), 201, {"Location": f"/books/{book['id']}"}
66     # UPDATE – PUT, DELETE – DELETE
67     @app.route("/books/<int:bid>", methods=["PUT", "DELETE"])
68     def modify_book(bid):
69         book = find(bid)
70         if not book:
71             return jsonify({"error": "not found"}), 404
72         if request.method == "PUT":
73             data = request.get_json(silent=True) or {}
74             # Nếu client cập nhật field year, kiểm tra tính hợp lệ
75             if "year" in data:
76                 y = data["year"]
77                 if not isinstance(y, int) or y < 1900:
78                     return (
79                         jsonify({"error": "year must be an integer >= 1900"}),
80                         400,
81                     )
82                 book["year"] = y
83             if "title" in data:
84                 book["title"] = data["title"]
85             if "author" in data:
86                 book["author"] = data["author"]
87             return jsonify(book), 200
88         BOOKS.remove(book)
89         return "", 204
90
91     if __name__ == "__main__":
92         app.run(host="127.0.0.1", port=5000, debug=True)

```